# Ders 20 Genişletilmiş Notu: Kruskal Algoritmasının Tasarımı ve Dijkstra ile En Kısa Yol

## Ders 20 Genişletilmiş Notu: Kruskal Algoritmasının Tasarımı ve Dijkstra ile En Kısa Yol

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Dersin Planı ve Açgözlü Algoritmalar
- 2. Kruskal Algoritması Üzerine Yeni Bir Örnek
- 3. Etiket Dizisi ve Dört Durum
- 4. Kruskal Algoritmasının Koda Dökülmesi
- 5. Kruskal'ın Karmaşıklığı ve Prim ile Karşılaştırılması
- 6. En Kısa Yol Problemi ve Dijkstra Algoritması
- 7. Dijkstra: Kaynaktan Hedefe Büyük Bir Örnek
- 8. Dijkstra: Öncelik Kuyruğu ve Tablo ile Küçük Bir Örnek
- 9. Gevşetme Adımı ve Dijkstra'nın Sözde Kodu
- 10. Renklendirme Üzerine Bir Soru ve Kullanım Alanları
- Örnek Sorular


### 1. Dersin Planı ve Açgözlü Algoritmalar

Hoca derse, bir önceki haftada kalınan yerden, yani minimum yayılan ağaç (minimum spanning tree, MST) konusundan devam edileceğini söyleyerek başladı. Dersin iki parçası vardı: önce Kruskal algoritmasının nasıl tasarlanıp koda döküleceği, sonra bir graf üzerinde en kısa yolun (shortest path) bulunması.

Hoca, graf üzerinde gezinme algoritmalarının, yani genişlik öncelikli arama (breadth-first search, BFS) ve derinlik öncelikli arama (depth-first search, DFS) algoritmalarının normalde ikinci yılda Veri Yapıları ve Algoritmalar dersinde anlatıldığını belirtti. Bu dönem eğitim uzaktan yürüdüğü için bu iki algoritma bir sonraki döneme bırakıldı.

Geçen hafta Prim ve Kruskal algoritmaları konuşulmuştu. Hoca ikisinin de açgözlü (greedy) algoritma olduğunu hatırlattı. Açgözlü algoritmalardaki mantık şudur: yerelde (lokalde) yapılan en iyi seçim, globalde de en iyi sonucu verir.

Amaç, bütün düğümleri birbirine bağlayan, döngü (cycle) oluşturmayan, ağaç yapısında olan ve toplam ağırlığı en küçük olan bir alt graf elde etmektir. İki algoritma bu amaca farklı yollardan gider:

- **Prim:** Herhangi bir düğümden başlar. En kısa mesafeleri ekleye ekleye sonuca ulaşır. Yerelde en iyi olan en kısa bağlantıyı seçmek, globalde de en iyi çözümü verir.
- **Kruskal:** Önce bütün bağlantıları (kenarları) ağırlığa göre küçükten büyüğe sıralar. Sonra en küçükten başlayarak, döngü oluşturmayan kenarları ala ala ilerler.

Kruskal'ın çözümüne geçen hafta başlanmış, süre yetmediği için yarıda kalmıştı. Bu derste algoritmanın nasıl tasarlanacağı anlatıldı. Hoca, bunu göstermeden önce hatırlatma amacıyla yeni bir graf üzerinde bir örnek daha yaptı.

## 2. Kruskal Algoritması Üzerine Yeni Bir Örnek

### 2.1 Kenarların Sıralanması

Hoca A'dan H'ye 8 düğümlü bir graf çizdi. Kruskal'da ilk iş kenarları küçükten büyüğe dizmektir. Mantık şuna dayanır: En küçükten başlayarak bağlantıları alırsak, globalde de en küçük toplamı, yani minimum yayılan ağacı elde etmiş oluruz.

Grafın kenarları ağırlığa göre sıralı olarak şöyledir (harf çifti kenarı, sayı ağırlığı gösterir):

`AB4, CF5, AC6, GH7, CD8, EG9, DF10, FG11, FH14, AD16, EF18, DH21, CE23, BE24`

**Soru:** Listeyi yazarken hoca son kenarı önce "DE24" diye okudu. Bir öğrenci sonuncusunun BE olması gerektiğini söyledi. **Cevap:** Hoca uyarıyı doğruladı. Grafta DE kenarı yoktur. Son iki kenar CE23 ve BE24'tür.

Hoca, kenarları küçükten büyüğe sıraladığımız için Kruskal'ın karmaşıklığının bu sıralamadan geldiğini söyledi (bkz. Bölüm 5).

### 2.2 Etiket Dizisi ile Adım Adım İlerleme

Algoritmik düşünmeye başlamak için her düğümün bir dizinin bir indisine denk geldiği düşünülür: A sıfırıncı, B birinci, …, H yedinci indistir. Bu dizi bir **etiket dizisi** (label array) olarak kullanılır. Başlangıçta hiçbir düğüm hiçbir yere bağlı değildir, bu yüzden dizinin bütün elemanları 0'dır. Kenarlar alındıkça her ayrı alt graf ayrı bir etiketle işaretlenir.

Aşağıdaki tabloda etiket dizisi A, B, C, D, E, F, G, H sırasıyla yazılmıştır.

| Kenar | Etiketler (önce) | Karar | Etiket dizisi (sonra) |
|---|---|---|---|
| AB4 | A=0, B=0 | İkisi de bağlı değil: yeni etiket 1 | 1 1 0 0 0 0 0 0 |
| CF5 | C=0, F=0 | İkisi de bağlı değil: yeni etiket 2 | 1 1 2 0 0 2 0 0 |
| AC6 | A=1, C=2 | Farklı alt graflar: birleştir, bütün 2'ler 1 olur | 1 1 1 0 0 1 0 0 |
| GH7 | G=0, H=0 | İkisi de bağlı değil: yeni etiket 3 | 1 1 1 0 0 1 3 3 |
| CD8 | C=1, D=0 | D, C'nin etiketini alır | 1 1 1 1 0 1 3 3 |
| EG9 | E=0, G=3 | E, G'nin etiketini alır | 1 1 1 1 3 1 3 3 |
| DF10 | D=1, F=1 | Aynı alt graf: alınmaz | 1 1 1 1 3 1 3 3 |
| FG11 | F=1, G=3 | Farklı alt graflar: birleştir, bütün 3'ler 1 olur | 1 1 1 1 1 1 1 1 |

Adımların hocanın anlatımındaki ayrıntıları şöyledir:

- **AB4:** A'da 0, B'de 0 var. İkisi de hiçbir yere bağlı değil, bu yüzden birbirlerine bağlanabilirler. İkisi aynı etiketle (1) etiketlenir.
- **CF5:** C de F de 0'dır. Bu kenar başka bir yerdedir, A-B ile bağlantısı yoktur. Bu yüzden C ile F farklı bir etiketle (2) etiketlenir, çünkü artık başka bir alt graf oluşmuştur.
- **AC6:** Burada bir alt graf (A-B) ve başka bir alt graf (C-F) vardır. Bunu etiketlerden anlarız: A'nın değeri 1, C'nin değeri 2. İkisi de bir yerlere bağlı ama farklı yerlere bağlı. O halde bu iki düğüm birbirine bağlanabilir. AC alındığında C-F de A-B'ye bağlanmış olur. Bu, bütün 2'leri 1 yaparak sağlanır. Hoca bütün 1'leri 2 yapmanın da hiç fark etmeyeceğini söyledi.
- **GH7:** G'de 0, H'de 0 var. Yeni bir etiketle (3) etiketlenirler.
- **CD8:** C'de 1, D'de 0 var. D aynı alt grafa girer, 1 yapılır.
- **EG9:** E'de 0, G'de 3 var. E bu alt grafa bağlanır, E 3 yapılır.
- **DF10:** D'de 1, F'de 1 var. İkisi zaten aynı alt graftadır, yani zaten birbirlerine bağlıdırlar (C üzerinden). Bu kenar bağlanırsa bir döngü oluşur. Ayrıca MST'deki hedef, bütün düğümlerin birbirine bağlı olduğu yapıyı minimum bağlantıyla kurmaktır. D ve F zaten

bağlıyken araya bir de 10 uzunluklu bağlantı eklemek gereksizdir. Bu yüzden DF10 alınmaz.
- **FG11:** F'de 1, G'de 3 var. Bir yanda A, B, C, D, F alt grafı, öbür yanda E, G, H alt grafı vardır. FG bağlanınca E ve H de aynı grafa bağlanır. Bütün 3'ler 1 yapılarak iki alt graf birleştirilir.

FG11 alındıktan sonra birbirine bağlı olmayan düğüm kalmaz. 8 düğüm için 7 kenar alınmıştır, işlem biter.

### 2.3 Toplam Ağırlık

Hoca toplamı önce yanlış saydı ("5, 9, 15, … 60 mı oldu?"), sonra düzeltti. Alınan kenarların ağırlıkları biriktirilerek toplanır:

$$4 \to 4+5=9 \to 9+6=15 \to 15+7=22 \to 22+8=30 \to 30+9=39 \to 39+11=50$$

Toplam ağırlık (total weight) 50'dir. Alınan kenarlar AB4, CF5, AC6, GH7, CD8, EG9, FG11'dir.

## 3. Etiket Dizisi ve Dört Durum

Hoca, geçen hafta konuşulan kuralları algoritmayı yazmadan önce bir köşeye not aldı. Bir kenarın iki ucundaki düğümlerin etiketlerine bakılır:

- **İkisi de 0:** İki düğüm de hiçbir yere bağlı değildir. Kenar alınır, iki düğüm yeni bir etiketle etiketlenir.
- **Biri 0, biri x:** Biri bir alt grafa bağlı, öbürü bağlı değildir. Kenar alınır, 0 olan düğüm x etiketiyle etiketlenir.
- **Biri x, biri y:** İki düğüm farklı alt graflardadır. Kenar alınır, bütün y etiketleri x yapılır (ya da tam tersi, hiç fark etmez).
- **İkisi de x:** İki düğüm zaten aynı alt graftadır. Kenar alınırsa döngü oluşur, bu yüzden alınmaz.

## 4. Kruskal Algoritmasının Koda Dökülmesi

### 4.1 Veri Yapısı

Önce bir kenar yapısı (structure) tanımlanır. Her kenarda bir U düğümü, bir V düğümü ve bunlar arasındaki ağırlık (weight) değeri vardır. Hoca önce akış şeması (flowchart) çizmeyi düşündü, ama yapı (struct) kullanıldığı için akıştan koda geçerken neyin ne olduğunun karışabileceğini söyleyerek doğrudan kod şeklinde yazdı.

Fonksiyonun girdileri şunlardır:

- Kenar tipinde bir graf dizisi (`gr`): hangi U düğümünün hangi V düğümüne hangi ağırlıkla bağlı olduğunu tutar. Kenarlar bu dizide ağırlığa göre küçükten büyüğe sıralıdır.
- Sonucun saklanacağı, yine kenar tipinde bir `mst` dizisi.
- Düğüm sayısı `nV`.

### 4.2 Başlangıç ve Döngü Koşulu

İlk etiket numarası (`labelNo`) 1'dir, çünkü 0 "etiketsiz" anlamına gelir. `nV` tane düğüm olduğu için `nV` elemanlı bir etiket dizisi vardır ve başlangıçta bütünüyle sıfırlanır. Örneğin 4 düğüm varsa 4 elemanlı bir etiket dizisi vardır. Ayrıca MST dizisinin indisi `i` ve graf dizisinin indisi `j` sıfırlanır.

Döngünün kaç kez döneceğini belirlemek için hoca şunu sordu: Ne kadar döneceğim? İki düğüm varsa aralarında bir bağlantı vardır. Üç düğüm varsa iki bağlantı vardır. Dolayısıyla bir minimum yayılan ağaç için düğüm sayısının bir eksiği kadar kenar gerekir. Bu yüzden döngü `i < nV - 1` olduğu sürece döner.

### 4.3 Döngünün Gövdesi

Her adımda graf dizisinin `j`. elemanının iki ucu alınır: `uu = gr[j].u`, `vv = gr[j].v`. Sonra durumlara bakılır.

**Birinci durum: ikisi de 0.** Hoca bunu `label[uu] + label[vv] == 0` koşuluyla kontrol etti. Etiketler negatif olmadığından toplam ancak ikisi de 0 ise sıfırdır. Bu durumda kenar doğrudan MST'ye alınır: `mst[i].u = uu`, `mst[i].v = vv`, `mst[i].weight = gr[j].weight`. Etiket dizisinde iki düğüm o andaki `labelNo` değeriyle etiketlenir ve `i` bir artırılır.

Hoca bunu bir örnekle gösterdi: 0. düğüm ile 5. düğüm arasında ağırlığı 3 olan bir kenar olsun. Etiket dizisinde 0 ile 5 yan yana 1 yapılır. MST'nin 0. elemanının U değeri 0, V değeri 5, ağırlığı 3 olur. Böylece MST'ye bir kenar eklenmiş olur.

**İkinci durum: etiketler farklı.** `label[uu] != label[vv]` ise iki düğüm farklı yerlerdedir ve kenar MST'ye alınmalıdır. Örneğin 0 ile 5 zaten bağlıyken, şimdi 0 ile 2'yi bağlamaya karar veriliyor olsun. MST'ye U olarak 0, V olarak 2 yazılır, ağırlık olarak `gr[j].weight` alınır ve `i` bir artırılır.

Ardından etiketin neden farklı olduğuna bakılır. Hoca bunun iki sebebi olabileceğini söyledi:

- Biri 0'dır. Örneğin x'in etiketi 2, y'nin etiketi 0'dır.
- İkisi de sıfırdan farklı ama farklı değerlerdir. Örneğin x'in etiketi 2, y'nin etiketi 3'tür.

Önce birinin 0 olup olmadığı kontrol edilir. `label[uu]` 0 ise yapılacak tek şey `label[uu] = label[vv]` diyerek onu öbürünün etiketiyle etiketlemektir. `label[vv]` 0 ise `label[vv] = label[uu]` yapılır. Hoca burada C'deki `!` (not) işlecini hatırlattı: `if (!label[vv])`, "etiket 0 ise" anlamına gelir. İkisi de 0 değilse iki farklı alt graf birleşiyordur. Bu iş ayrı bir `union` fonksiyonuna bırakılır.

**Üçüncü durum: etiketler eşit.** Bununla ilgilenilmez. Hangi durum olursa olsun döngünün sonunda `j++` yapılır, yani graftaki bir sonraki kenara geçilir. Etiketleri eşit olan bir kenar böylece atlanmış olur.

### 4.4 Birleştirme (union) Fonksiyonu

Hoca iki alt grafın birleşmesini yine bir örnekle açıkladı. A, B, C bağlanmış ve 1 ile etiketlenmiş, G ile H bağlanmış ve 3 ile etiketlenmiş olsun. Bunlar iki ayrı alt graftır. C-G bağlantısı gelince bunlar tek bir alt graf olur. Bu anda G'nin ve H'nin ikisinin birden 1 olması gerekir, çünkü artık aynı alt grafın içindedirler.

Bunu sağlamak çok basittir. Fonksiyon etiket dizisini, düğüm sayısını ve bağlanacak iki tarafı alır. Etiket dizisinde değeri birinci tarafın etiketine eşit olan her eleman, ikinci tarafın etiketine çevrilir. Hoca yönün fark etmediğini vurguladı: Birinin etiketi 1, öbürünün 3 ise ister bütün 1'ler 3 yapılır, ister bütün 3'ler 1 yapılır.

### 4.5 Hocanın Yazdığı Kod

Hocanın tablete yazdığı kod şöyledir:

```
typedef struct Edge {
    int u, v;
    int weight;
} EDGE;

void kruskal(EDGE gr[], EDGE mst[], int nV)
{
    int labelNo = 1, i, j;
    for (i = 0; i < nV; i++)
        label[i] = 0;
    i = j = 0;
    while (i < nV - 1) {
        uu = gr[j].u;
        vv = gr[j].v;
```

```
        if (label[uu] + label[vv] == 0) {   /* ikisi de etiketsiz */
            mst[i].u = uu;
            mst[i].v = vv;
            mst[i].weight = gr[j].weight;
            label[uu] = label[vv] = labelNo++;
            i++;
        }
        if (label[uu] != label[vv]) {       /* farklı alt graflar */
            mst[i].u = uu;
            mst[i].v = vv;
            mst[i].weight = gr[j].weight;
            i++;
            if (!label[uu])
                label[uu] = label[vv];
            if (!label[vv])
                label[vv] = label[uu];
            else
                union(label, nV, uu, vv);
        }
        j++;                                /* etiketler eşitse kenar atlanır */
    }
}

void union(int label[], int nV, int uu, int vv)
{
    int i;
    for (i = 0; i < nV; i++)
        if (label[i] == uu)
            label[i] = vv;
}
```

**Ders sonrası not:** Bu kod olduğu gibi derlenmez ve düzeltilse bile yanlış sonuç verir. Hatalar şunlardır:

- `union` C'de ayrılmış bir anahtar sözcüktür (`struct` gibi bir tür bildirir). `gcc -Wall` ile derlendiğinde `void union(...)` satırı "expected '{' before '(' token" hatası verir. Fonksiyona başka bir ad (örneğin `birlestir`) verilmelidir.
- Fonksiyon içinde `label[i] == uu` karşılaştırması yapılıyor. Oysa `uu` ve `vv` düğüm indisleridir, etiket değeri değildir. Hocanın sözlü anlatımı ("label UU değerine sahipse onun değerini VV yap") ve tahtadaki örneği (1 ile 3) etiket değerleri üzerindendir. Doğrusu fonksiyona `label[uu]` ve `label[vv]` değerlerini göndermektir. Değerler önceden saklanmalıdır, yoksa döngü sırasında `label[uu]` değişir ve kalan elemanlar yanlış karşılaştırılır.
- Kodu indisle karşılaştıran haliyle bu dersteki A-H örneğinde python ile çalıştırınca AC6 adımında etiketi 0 olan bütün düğümler 2 yapılır. Sonuç 7 yerine 5 kenarlı, toplam ağırlığı 55 olan yanlış bir kümedir (AB4, CF5, AC6, AD16, BE24).
- İkinci `if (!label[vv])` aslında `else if` olmalıdır. Mevcut halde `label[uu]` 0 iken ona etiket verildikten sonra `else` kolu da çalışır ve gereksiz bir birleştirme çağrılır.
- `label` dizisi ile `uu`, `vv` değişkenleri tanımlanmamıştır. Ayrıca döngüde `j`'nin kenar sayısını aşıp aşmadığı kontrol edilmez. Graf bağlı değilse `nV - 1` kenar hiçbir zaman bulunamaz ve `gr` dizisinin dışına taşılır.

Bu düzeltmeler uygulanmış, derste çizilen A-H grafı üzerinde çalışan tam program aşağıdadır. `gcc -Wall -Wextra` ile uyarısız derlenmiş ve çalıştırılmıştır (A=0, B=1, ..., H=7):

```
#include <stdio.h>

typedef struct Edge {
    int u, v;
    int weight;
} EDGE;
```

```c
/* eski etiketli bütün düğümleri yeni etikete taşır */
void birlestir(int label[], int nV, int eski, int yeni)
{
    int i;
    for (i = 0; i < nV; i++)
        if (label[i] == eski)
            label[i] = yeni;
}

/* gr: ağırlığa göre küçükten büyüğe sıralı nE kenar; döndürülen: MST kenar sayısı */
int kruskal(EDGE gr[], int nE, EDGE mst[], int nV)
{
    int label[nV];
    int labelNo = 1, i, j, uu, vv;
    for (i = 0; i < nV; i++)
        label[i] = 0;
    i = j = 0;
    while (i < nV - 1 && j < nE) {
        uu = gr[j].u;
        vv = gr[j].v;
        if (label[uu] != label[vv] || label[uu] == 0) {
            mst[i++] = gr[j];
            if (label[uu] == 0 && label[vv] == 0)       /* ikisi de 0: yeni etiket */
                label[uu] = label[vv] = labelNo++;
            else if (label[uu] == 0)                    /* biri 0, biri x */
                label[uu] = label[vv];
            else if (label[vv] == 0)
                label[vv] = label[uu];
            else                                        /* biri x, biri y */
                birlestir(label, nV, label[vv], label[uu]);
        }
        j++;                                            /* ikisi de x: kenar atlanır */
    }
    return i;
}

int main(void)
{
    /* A=0, B=1, ..., H=7 */
    EDGE gr[] = {
        {0,1,4}, {2,5,5}, {0,2,6}, {6,7,7}, {2,3,8}, {4,6,9}, {3,5,10},
        {5,6,11}, {5,7,14}, {0,3,16}, {4,5,18}, {3,7,21}, {2,4,23}, {1,4,24}
    };
    int nE = sizeof gr / sizeof gr[0], nV = 8, k, i, toplam = 0;
    EDGE mst[7];
    k = kruskal(gr, nE, mst, nV);
    for (i = 0; i < k; i++) {
        printf("%c%c%d\n", 'A' + mst[i].u, 'A' + mst[i].v, mst[i].weight);
        toplam += mst[i].weight;
    }
    printf("kenar sayisi: %d, toplam agirlik: %d\n", k, toplam);
    return 0;
}
```

Çıktı:

```
AB4
CF5
AC6
GH7
```

```
CD8
EG9
FG11
kenar sayisi: 7, toplam agirlik: 50
```

Programdaki `birlestir(label, nV, label[vv], label[uu])` çağrısı, `vv` tarafının etiketini `uu` tarafının etiketine çevirir. Bu, hocanın tahtadaki yönüyle aynıdır: AC6'da bütün 2'ler 1, FG11'de bütün 3'ler 1 olur. `label[vv]` değeri çağrı anında kopyalanarak geçtiği için döngü sırasında değişmez.

## 5. Kruskal'ın Karmaşıklığı ve Prim ile Karşılaştırılması

Hoca, Kruskal'da kenarlar sıralandığı için karmaşıklığın bu sıralamadan geldiğini söyledi. $V$ tane düğüm ve $E$ tane kenar olan bir grafta $E$ tane kenar sıralanır. Sıralama karmaşıklığı $n \log n$ biçiminde olduğundan Kruskal'ın karmaşıklığı $O(E \log V)$ olarak verilir.

> **Ek bilgi (derste anlatılmadı):** $E$ kenarı sıralamak $O(E \log E)$ sürer. Basit bir grafta $E \leq V^2$ olduğundan $\log E \leq 2 \log V$ olur, bu yüzden $O(E \log E) = O(E \log V)$ yazılabilir. Derste söylenen $O(E \log V)$ bu eşitlikten gelir.
>
> **Ders sonrası not:** $O(E \log V)$ yalnız sıralamanın maliyetidir. Bu derste yazılan etiket dizisi yönteminde her birleştirme bütün etiket dizisini taradığı için $O(V)$ sürer. En fazla $V - 1$ birleştirme olduğundan bu kısım en kötü durumda $O(V^2)$ tutar. Toplam maliyet bu gerçeklemede $O(E \log V + V^2)$ olur.
>
> **Ek bilgi (derste anlatılmadı):** Kruskal'ın $O(E \log V)$ sınırında kalması için etiket dizisi yerine ayrık küme (disjoint-set, union-find) yapısı kullanılır. Bu yapıda her küme bir ağaçla temsil edilir. Rütbeye göre birleştirme (union by rank) ve yol sıkıştırma (path compression) ile bir düğümün hangi kümede olduğunu bulmak ve iki kümeyi birleştirmek neredeyse sabit sürede yapılır.

Hoca iki algoritmayı şöyle karşılaştırdı:

- Çok fazla bağlantının olduğu (yoğun, dense) bir grafta Prim daha başarılıdır, daha kısa sürede çalışır.
- Düğüm sayısı çok ama bağlantıları az olan seyrek (sparse) bir grafta Kruskal daha iyi çalışır.
- Prim'in sıkıntısı, bir düğüme durmadan birden fazla kez bakma ihtiyacından kaynaklanır. Prim her adımda bağlı düğümlerin içinden en kısa bağlantıyı bulmak zorunda olduğu için bu düğümleri tekrar tekrar kontrol eder.
- Kruskal'da böyle bir şey yoktur. A ile B'yi bağlayan bir kenarla iş bittiyse bir daha dönüp A ile B'ye bakmaya gerek kalmaz.
- Hoca ağ bağlantısı uygulamalarında genelde Kruskal kullanıldığını ekledi.

## 6. En Kısa Yol Problemi ve Dijkstra Algoritması

Aradan sonra hoca en kısa yol bulmaya geçti. En kısa yolu bulmanın birden fazla yolu vardır. Bunun genişlik öncelikli arama ile yapılan bir hali de vardır, ama BFS bir sonraki döneme bırakıldığı için hoca Dijkstra algoritmasını Prim'e benzer bir şekilde gösterdi.

Prim algoritması minimum yayılan ağacı elde etmek için kullanılır. Dijkstra buna çok benzer bir şekilde en kısa yolu elde etmek için kullanılır. İkisinde de aynı mantık vardır: Bir başlangıç düğümünden en küçük bağlantıları bularak ilerlemek.

Hocanın verdiği temel bilgiler şunlardır:

- Dijkstra algoritması hem yönlü hem yönsüz graflarda kullanılabilir.
- Kenar ağırlıklarının pozitif olması gerekir.
- Hoca, Dijkstra'nın bu algoritmayla 1972'de Turing Ödülü'nü (Turing Award) kazandığını ve 1972'de ortaya konmasına rağmen algoritmanın hâlâ kullanıldığını söyledi.

> **Ders sonrası not:** Dijkstra'nın doğruluğu için gereken koşul, kenar ağırlıklarının negatif olmamasıdır. 0 ağırlıklı kenarlar sorun çıkarmaz, çünkü kuyruktan çıkan düğümün mesafesi sonradan küçülemez. Negatif ağırlıklı kenar varsa kuyruktan çık-

mış bir düğüme sonradan daha kısa bir yol bulunabilir ve algoritma yanlış sonuç verebilir.

**Ders sonrası not:** Hoca algoritmanın 1972'de ortaya konduğunu ve ödülün bu algoritma için verildiğini söyledi. Algoritma büyük olasılıkla 1956'da tasarlanmış ve 1959'da yayımlanmıştır. Dijkstra 1972 Turing Ödülü'nü büyük olasılıkla tek bir algoritma için değil, programlama dillerine ve yapısal programlamaya genel katkıları için almıştır.

Problem şudur: Verilen bir grafta bir A düğümünden bir B düğümüne en kısa yol nasıl bulunur?

## 7. Dijkstra: Kaynaktan Hedefe Büyük Bir Örnek

### 7.1 Graf ve Başlangıç

Hoca "bayağı karmaşık" bir yönlü graf çizdi. Bir S düğümü kaynak (source), bir T düğümü hedeftir (target). Amaç S'den T'ye gitmektir. Grafın kenarları şöyledir:

```
S→2: 9    S→6: 14   S→7: 15
2→3: 24
6→3: 18   6→5: 30   6→7: 5
7→5: 20   7→T: 44
3→5: 2    3→T: 19
5→4: 11   5→T: 16
4→3: 6    4→T: 6
```

Başlangıçta herhangi bir düğüme gelmek için gereken mesafe sonsuz kabul edilir. Örneğin 3. düğüme 2 üzerinden de 6 üzerinden de gelinebilir. İşe ilk başlarken hangi düğüme nereden gelmenin en kısa olacağı bilinmez. Bu yüzden bütün düğümlere ulaşma mesafesi olarak sonsuz, pratikte olabilecek en büyük değer (`MAXINT`) atanır. Bir tek kaynağın mesafesi 0'dır, çünkü kaynağın kendisine olan mesafesi 0'dır.

### 7.2 Adım Adım Çözüm

Hoca Prim mantığıyla, her adımda o ana kadar bulunan adaylardan en kısa mesafeli olanı seçerek ilerledi.

**Adım 1.** Kaynaktan çıkılabilecek en kısa mesafe S→2 kenarıdır: 9. Kaynaktan çıkınca bundan daha kısa bir şey yapılamayacağı için önce 2'ye gidilir. 2'nin mesafesi artık sonsuz değil, 9'dur.

**Adım 2.** 2'den gidilebilecek yol 3'e 24'tür ($9 + 24 = 33$). Kaynaktan hâlâ 14 ve 15 uzunluklu kenarlar vardır. En kısa olan S→6 alınır. 6'nın mesafesi 14 olur.

**Adım 3.** 6 işin içine girince 5, 30 ve 18 ağırlıklı kenarlar devreye girer:

- 7'ye 6 üzerinden: $14 + 5 = 19$. Ama 7'ye kaynaktan doğrudan 15 ile gidilebilir.
- 3'e 6 üzerinden: $14 + 18 = 32$. 2 üzerinden 33'tür.
- 5'e 6 üzerinden: $14 + 30 = 44$.

O anki en kısa aday kaynaktan 7'ye 15'tir. 7 alınır, mesafesi 15 olur.

**Adım 4.** Şimdi 2, 6 ve 7 alınmıştır. Adaylar şunlardır:

| Hedef düğüm | Aday yollar | En iyi aday |
|---|---|---|
| 3 | 2 üzerinden $9 + 24 = 33$, 6 üzerinden $14 + 18 = 32$ | 32 |
| 5 | 6 üzerinden $14 + 30 = 44$, 7 üzerinden $15 + 20 = 35$ | 35 |
| T | 7 üzerinden $15 + 44 = 59$ | 59 |

Bu adaylardan en küçüğü 32'dir. 3 alınır, mesafesi 32 olur.

**Soru:** Hoca bu adımda bir an 3'e 6 üzerinden gidilemeyeceğini söyledi. Bir öğrenci 3'e 6 üzerinden gidince 32 olduğunu hatırlattı. **Cevap:** Hoca hatasını kabul etti: 6'dan 3'e 18'lik bir kenar vardır, $14 + 18 = 32$ eder. 2 üzerinden gidilince $9 + 24 = 33$ olur. 3'e 6 üzerinden 32 ile gelinir.

**Adım 5.** 3 alındığına göre 3'ten gidilebilecek yerlere bakılır:

- 5'e: 3→5 kenarı 2 olduğu için $32 + 2 = 34$. 7 üzerinden 35 idi.
- T'ye: $32 + 19 = 51$. 7 üzerinden 59 idi.
- 4'e şu an için gidilemez, 4'e doğrudan gelen bir bağlantı yoktur.

En küçük aday 34'tür. 5'e 3 üzerinden gelmek kabul edilir, 5'in mesafesi 34 olur.

**Adım 6.** 5'ten yeni aday mesafeler:

- 4'e: $34 + 11 = 45$.
- T'ye: $34 + 16 = 50$.

Adaylar 45 (4) ve 50 (T)'dir. Mantıklısı 5'ten 4'e gitmektir. 4'ün mesafesi 45 olur.

**Adım 7.** 4'ten T'ye gelinmeye kalkılırsa $45 + 6 = 51$ eder. Halbuki 5'ten T'ye 16'lık bir kenar vardır ve 5'ten gelinirse $34 + 16 = 50$ eder. 4'ten 3'e gidiş de $45 + 6 = 51$ eder ve 3'ün 32'lik mesafesinden büyüktür (ara adım eklendi). T'ye 5'ten 50 ile gelinir.

### 7.3 Sonuç

Her düğüm için bulunan en kısa mesafe ve gelinen düğüm şöyledir:

| Seçilme sırası | Düğüm | Mesafe | Nereden |
|---|---|---|---|
| 1 | 2 | 9 | S |
| 2 | 6 | 14 | S |
| 3 | 7 | 15 | S |
| 4 | 3 | 32 | 6 ($14 + 18$) |
| 5 | 5 | 34 | 3 ($32 + 2$) |
| 6 | 4 | 45 | 5 ($34 + 11$) |
| 7 | T | 50 | 5 ($34 + 16$) |

En kısa yol S → 6 → 3 → 5 → T'dir ve toplam uzunluğu $14 + 18 + 2 + 16 = 50$'dir. Bu sıra ve bu mesafeler, ikili yığın (binary heap) kullanan bir Dijkstra gerçeklemesiyle python'da doğrulanmıştır.

## 8. Dijkstra: Öncelik Kuyruğu ve Tablo ile Küçük Bir Örnek

Kısa bir aradan sonra hoca, algoritmanın nasıl çözüldüğünü daha küçük bir graf üzerinden adım adım gösterdi ve algoritmayı bu graf üzerinden geliştirdi.

### 8.1 Graf ve Komşuluk Listesi

Graf yönlüdür. Düğümler S (kaynak), A, B, C ve D'dir. Bağlantılar şöyledir: S'den B'ye 7, S'den A'ya 2, A'dan B'ye 3, B'den A'ya 2, B'den C'ye 1, A'dan C'ye 8, A'dan D'ye 5, C'den D'ye 4 ve D'den C'ye 5. Hoca bu grafın komşuluk listesini (adjacency list) birlikte çıkardı:

```
S: A(2), B(7)
A: B(3), C(8), D(5)
B: A(2), C(1)
C: D(4)
D: C(5)
```

Algoritma ilerlerken bir düğümden nerelere gidildiğine hep bu komşuluk listesinden bakılır.

### 8.2 Başlangıç (Initialization)

Her düğüm için üç bilgi tutulur:

- **Mesafe (distance, `d`):** Kaynağa olan uzaklık.
- **Önceki düğüm (`pred`):** Bu mesafeye hangi düğümden gelindiği.
- **Renk (`color`):** Beyaz düğüm henüz seçilmemiş, siyah düğüm seçilmiş demektir.

İşe ilk başlandığında kaynağın kendisine olan mesafesi 0, diğer bütün düğümlerin mesafesi sonsuzdur. Hiçbir düğüme nereden gelindiği henüz bilinmediği için `pred` değerleri NULL'dır.

Hoca bu problemin bir öncelik kuyruğu (priority queue) ile çözüldüğünü söyledi. Kısa mesafeleri tutmamız gerektiği için mantıklı olan bunları bir kuyrukta tutarak ilerlemektir. Mesafeler sürekli değişip kısalacağı için bu kuyruk sıradan bir kuyruk değil, öncelik kuyruğudur. Başlangıçta bütün düğümler bu kuyruğa konur.

### 8.3 Adım Adım Çözüm

**S çıkar.** Kuyruğun başında mesafesi 0 olan S vardır. Yola S'den çıkıldığı için S siyah yapılır. S'den A'ya gidilince mesafe 2, B'ye gidilince mesafe 7 olur. A'ya da B'ye de S'den gelinmiş olur. Kuyrukta A (2), B (7), C ($\infty$), D ($\infty$) kalır.

**A çıkar.** Kuyruğun başında A vardır, mesafesi 2'dir. A siyah yapılır. A'nın bütün komşuları güncellenir (update):

- B: A üzerinden $2 + 3 = 5$. 7'den küçük olduğu için B'nin mesafesi 5, `pred` değeri A olur.
- C: $2 + 8 = 10$. C'nin mesafesi 10, `pred` değeri A olur.
- D: $2 + 5 = 7$. D'nin mesafesi 7, `pred` değeri A olur.

Kuyrukta B (5), C (10), D (7) kalır.

**B çıkar.** En kısa B olduğu için kuyruğun başında B vardır. B siyah yapılır. B'ye A üzerinden gelinmiştir, çünkü $2 + 3 < 7$. B'nin komşularına bakılır. B'den C'ye 1'lik bir bağlantı vardır: $5 + 1 = 6$. 10'dan küçük olduğu için C'nin mesafesi 6, `pred` değeri B olur. B'den A'ya da bir kenar vardır, ama A zaten kuyruktan çıkmıştır ve $5 + 2 = 7$, A'nın 2'lik mesafesinden büyüktür (ara adım eklendi). Kuyrukta C (6), D (7) kalır.

**C çıkar.** C siyah yapılır. C'den yalnız D'ye gidilir, ağırlık 4'tür. $6 + 4 = 10$ olur. Bu, D'nin 7'lik mesafesinden büyük olduğu için D'ye dokunulmaz. D'ye hâlâ A'dan gelinir.

**D çıkar.** Kuyrukta yalnız D (7) kalmıştır. Daha küçük bir bağlantı olmadığı için D de siyah yapılır. D'den C'ye $7 + 5 = 12$ ise C'nin 6'lık mesafesinden büyüktür (ara adım eklendi). D çıkınca kuyruk boşalır ve iş biter.

Adımların özeti:

| Adım | Kuyruktan çıkan | d[A] | d[B] | d[C] | d[D] |
|---|---|---|---|---|---|
| Başlangıç | - | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 1 | S (0) | 2 | 7 | $\infty$ | $\infty$ |
| 2 | A (2) | 2 | 5 | 10 | 7 |
| 3 | B (5) | 2 | 5 | 6 | 7 |
| 4 | C (6) | 2 | 5 | 6 | 7 |
| 5 | D (7) | 2 | 5 | 6 | 7 |

### 8.4 Son Durum ve Yolun Okunması

Hoca son halini temiz bir tabloya yazdı:

| Düğüm | S | A | B | C | D |
|---|---|---|---|---|---|
| d | 0 | 2 | 5 | 6 | 7 |
| pred | NULL | S | A | B | A |

Kaynağın kendisinden önce kimse yoktur, bu yüzden `pred` değeri NULL'dır. A'ya S'den, B'ye A'dan, C'ye B'den, D'ye A'dan gelinmiştir.

> **Ders sonrası not:** Hoca bir yerde "A'ya da D'den gelindi" dedi. Doğrusu "D'ye A'dan gelindi"dir ($2 + 5 = 7$). Hoca hemen ardından tabloya bunu doğru yazdı.

`pred` dizisi, herhangi bir düğüme en kısa yoldan nereden gelindiğini geriye doğru izlemeyi sağlar:

- B: S'den A'ya 2, A'dan B'ye 3; toplam 5.
- C: C'ye B'den gelinmiştir. S'den A'ya 2, A'dan B'ye 3, B'den C'ye 1; toplam 6.
- D: D'ye A'dan gelinmiştir. S'den A'ya 2, A'dan D'ye 5; toplam 7.

Hoca sonucu bir ağaç olarak da çizdi: kökte mesafesi 0 olan S, onun altında A (2), A'nın altında B (5) ve D (7), B'nin altında C (6).

```
S (0)
└── A (2)
    ├── B (5)
    │   └── C (6)
    └── D (7)
```

## 9. Gevşetme Adımı ve Dijkstra'nın Sözde Kodu

### 9.1 Hangi Yoldan Gelmeli?

Hoca algoritmanın özündeki kararı küçük bir örnekle açıkladı. Kaynak S, bir W düğümü ve bir V düğümü olsun. S'den W'ye doğrudan bağlantı 47, S'den V'ye 11, V'den W'ye 33 olsun. Soru şudur: W'ye doğrudan S'den mi gitmek iyidir, yoksa V üzerinden mi? V üzerinden toplam $11 + 33 = 44$ eder. 44, 47'den küçük olduğu için W'ye V üzerinden gidilir.

Demek ki her adımda kontrol edilecek şey şudur: S'den W'ye bilinen mesafe, S'den V'ye mesafe ile V'den W'ye kenarın toplamından büyükse, W'ye V'den gelinmelidir. Yapılan iş, hangi taraftan gelmenin daha iyi olduğunu anlamaktır.

### 9.2 Sözde Kod

Hoca algoritmayı, öğrencilerde bulunsun diye sözde kod (pseudo code) olarak yazdı. Girdiler G grafı, kaynak düğüm `s` ve ağırlık matrisi `w`'dir.

```
Dijkstra(G, s, w)
{
    for (each u ∈ V) {
        d[u] = ∞;
        color[u] = white;
    }
    d[s] = 0;
    pred[s] = NULL;
    PQ = (queue with all vertices)
    while (non-Empty(PQ)) {
        u = extract-Min(PQ)
        for (each v ∈ Adj[u]) {
            if (d[u] + w(u,v) < d[v]) {
                d[v] = d[u] + w(u,v);
                DECREASE-KEY(PQ, v, d[v]);
                pred[v] = u;
            }
        }
        color(u) = BLACK;
    }
}
```

Kodun parçaları şöyle açıklandı:

- **Başlangıç:** Bütün düğümlerin kaynağa mesafesi sonsuz yapılır. Renk yalnız işlenip işlenmediğini göstermek içindir; beyaz-siyah yerine 0-1 de kullanılabilir. Kaynağın kendisine mesafesi 0 yapılır. Kaynaktan önce kimse olmadığı için `pred[s]` NULL'dır. Sonra bütün düğümler kuyruğa yerleştirilir.
- **Kuyruktan çıkarma:** Öncelik kuyruğu olduğu için her seferinde o anda en küçük mesafeli düğüm çıkar (`extract-Min`), diğerleri durur. Hoca burada bir noktaya dikkat çekti. Küçük örnekte bir adım önce B'nin mesafesi 5, C'nin 10, D'nin 7 idi. Ama bütün kuyruk sıralanmaya çalışılmadı, çünkü bir adım sonra C'nin mesafesi 10'dan 6'ya düştü. Amaç

kuyruktan yalnız en küçük mesafeli, yani en öncelikli düğümü çıkarmaktır; diğerleriyle uğraşılmaz.

- **Gevşetme (relaxation):** Çıkan `u` düğümünün komşusu olan her `v` için mesafe hesabı yapılır. `u`'nun mesafesine `u`'dan `v`'ye olan ağırlık eklendiğinde `v`'nin bir önceki adımdaki mesafesinden küçük bir değer çıkıyorsa $d[v] = d[u] + w(u, v)$ yapılır.
- **Kuyruğun güncellenmesi:** `v`'nin mesafesi değiştiği için öncelik kuyruğundaki anahtarı (key) da güncellenir (`DECREASE-KEY`). Aynı zamanda `v`'ye artık `u`'dan gelinmiş olur: `pred[v] = u`.
- **Bitiş:** `u`'nun bütün komşuları işlendiğinde `u` siyah yapılır, yani `u` ile işimiz bitmiştir.

Hoca gevşetme adımını bir örnekle daha gösterdi. A'dan B'ye bağlantı 30, A'dan C'ye 15, C'den B'ye 8 olsun. C işin içine girdiğinde şuna karar verilir: B'nin A'ya olan mesafesi 30 olarak mı kalsın, yoksa A → C → B üzerinden 23 mü olsun? Bunun için $d[C] + w(C, B) = 15 + 8 = 23$ değerinin 30'dan küçük olup olmadığına bakılır. Küçük olduğu için bu bağlantı daha kârlıdır. B'nin mesafesi 30'dan 23'e iner, öncelik kuyruğundaki değeri de 23'e güncellenir ve B'ye artık A'dan değil C'den gelinir.

> **Ek bilgi (derste anlatılmadı):** Dijkstra'nın çalışma süresi öncelik kuyruğunun nasıl gerçeklendiğine bağlıdır. Kuyruk düz bir dizi olarak tutulursa her `extract-Min` $O(V)$ sürer ve toplam $O(V^2)$ olur. Kuyruk ikili yığın (binary heap) ile tutulursa `extract-Min` ve `DECREASE-KEY` $O(\log V)$ sürer; toplam $O((V + E) \log V)$ olur.

## 10. Renklendirme Üzerine Bir Soru ve Kullanım Alanları

Hoca, Dijkstra'nın çok kullanılan algoritmalardan biri olduğunu söyledi. Trafik planlamasında, robot navigasyonunda, ağ yönlendirme (network routing) protokollerinde ve zamanlama (scheduling) işlemlerinde hâlâ kullanılmaktadır.

**Soru:** Düğümleri siyaha boyuyoruz. Bu tam olarak ne için? **Cevap:** Bunun anlamı, o düğümün alındığını göstermektir. Aslında düğüm kuyruktan çıkarıldığı anda onunla iş bitmiş olur. Örneğin S çıkarıldı, işi bitti; A çıkarıldı, işi bitti; B çıkarıldı, işi bitti. Boyamak da boyamamak da olur. Tabloda kimin işlendiği gözüksün ve bir şey kaçırılmasın diye yapılır, yani görsel amaçlıdır. Algoritmik olarak ayrı bir dizi tutmaya ya da ayrı bir iş yapmaya gerek yoktur, çünkü yalnız öncelik kuyruğuna bakılır. Kuyruktan çıkan bir düğümle bir daha uğraşılmaz. Hoca bunun güzel bir soru olduğunu söyledi.

## Örnek Sorular

Bu derste hazır kare yoktu. Aşağıdaki sorular yalnız derste sözlü sorulan sorulardır.

**Soru 1** (derste sözlü, 00:14:59). A-H grafında kenarlar sırayla işlenirken AB4, CF5, AC6, GH7, CD8 ve EG9 alınmıştır. Sıradaki DF10 kenarı MST'ye alınmalı mı?

> **Çözüm:** Alınmamalı. Etiket dizisinde D'nin ve F'nin etiketi 1'dir, yani ikisi zaten aynı alt graftadır (D-C-F üzerinden bağlıdırlar). DF10 eklenirse bir döngü oluşur ve ağaca gereksiz yere 10 uzunluk eklenir.

**Soru 2** (derste sözlü, 00:22:04). Kruskal'ın döngüsü kaç kez dönmelidir? $n$ düğümlü bir grafın minimum yayılan ağacında kaç kenar vardır?

> **Çözüm:** İki düğüm için bir, üç düğüm için iki bağlantı gerekir. Genel olarak $n$ düğüm için $n - 1$ kenar vardır. Bu yüzden döngü `i < nV - 1` olduğu sürece döner. A-H örneğinde 8 düğüm için 7 kenar alınmıştır.

**Soru 3** (derste sözlü, 00:27:53). Bir kenarın iki ucundaki düğümlerin etiketleri farklıysa bu neden olmuş olabilir?

> **Çözüm:** İki sebep vardır. Ya biri 0'dır (örneğin x'in etiketi 2, y'nin etiketi 0); bu durumda 0 olan düğüm öbürünün etiketini alır. Ya da ikisi sıfırdan farklı ama farklı değerlerdir (örneğin 2 ve 3); bu durumda iki alt graf birleştirilir ve birinin bütün etiketleri öbürüne çevrilir. Her iki durumda da kenar MST'ye eklenir.

**Soru 4** (derste sözlü, 01:00:59). S-T grafında 2, 6 ve 7 alınmıştır. Adaylar şunlardır: 5'e gidiş 35, 3'e gidiş 32, T'ye gidiş 59. Hangisini almak doğrudur?

**Çözüm:** En küçük aday olan 32 alınır: 3'e 6 üzerinden $14+18=32$ ile gelinir. 2 üzerinden yol $9+24=33$'tür. Hoca bu adımda bir an 3'e 6 üzerinden gidilemeyeceğini söyledi; bir öğrencinin uyarısıyla (01:02:10) 32'nin 6 üzerinden geldiği düzeltildi. Devamında 5 (34), 4 (45) ve T (50) alınır; en kısa yol S → 6 → 3 → 5 → T'dir.

**Soru 5** (derste sözlü, 01:18:24). S'den W'ye doğrudan bağlantı 47, S'den V'ye 11, V'den W'ye 33'tür. W'ye hangi yoldan gidilmelidir?

**Çözüm:** V üzerinden gidilmelidir, çünkü $11+33=44<47$. Genel kural: $d[V]+w(V,W)<d[W]$ ise W'ye V'den gelinir.

**Soru 6** (derste sözlü, 01:34:36). A'dan B'ye bağlantı 30, A'dan C'ye 15, C'den B'ye 8'dir. C alındıktan sonra B'nin mesafesi 30 olarak mı kalmalı, yoksa A → C → B üzerinden mi hesaplanmalı?

**Çözüm:** $15+8=23<30$ olduğu için B'nin mesafesi 23'e iner. Öncelik kuyruğunda B'nin anahtarı 23'e güncellenir (`DECREASE-KEY`) ve `pred[B] = C` olur.

**Soru 7** (derste sözlü, 01:37:28). Bir öğrenci sordu: Dijkstra'da düğümleri siyaha boyamak tam olarak ne içindir?

**Çözüm:** Hoca, boyamanın yalnız düğümün işlendiğini göstermek için, görsel amaçlı yapıldığını söyledi. Düğüm öncelik kuyruğundan çıktığı anda onunla iş biter ve bir daha uğraşılmaz. Bu yüzden algoritmik olarak ayrı bir renk dizisi tutmak gerekmez.

## Dersin Toparlaması

Bu derste önce Kruskal algoritması, kenarların küçükten büyüğe sıralanması ve bir etiket dizisiyle hangi düğümün hangi alt grafa ait olduğunun izlenmesi üzerinden tasarlandı. Bir kenarın iki ucunun etiketlerine bakılarak dört durum ayırt edildi: ikisi de 0 ise yeni etiket, biri 0 ise öbürünün etiketi, ikisi farklı ise iki alt grafın birleştirilmesi, ikisi aynı ise kenarın döngü oluşturacağı için atlanması. A-H örneğinde bu kurallarla toplam ağırlığı 50 olan 7 kenarlı ağaç bulundu. Kruskal'ın karmaşıklığının sıralamadan geldiği, seyrek graflarda Kruskal'ın, yoğun graflarda Prim'in daha iyi çalıştığı söylendi.

Dersin ikinci yarısında en kısa yol problemi için Dijkstra algoritması işlendi. Algoritma Prim'e benzer biçimde kaynaktan başlayıp her adımda öncelik kuyruğundan en küçük mesafeli düğümü çıkarır ve komşularının mesafesini $d[u]+w(u,v)<d[v]$ koşuluyla günceller. `pred` dizisi en kısa yolun geriye doğru okunmasını sağlar. Hoca bu dersle graflar konusunun bittiğini, bir sonraki hafta metin arama (string search) konusuna geçileceğini söyledi.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.